EF Core: Include() + N+1 Problem

Loading related data | Concise lesson summary

Core idea

Navigation properties are not automatically populated just because the main entity was queried. When related data is needed, load it intentionally. Include() is one option for eager-loading related entities; projection is another option when only selected values are needed.

Mental anchor: A database call inside a loop is a warning sign for the N+1 problem.

What is the N+1 problem?

The application first executes one query to load the main records, then executes another query for each record to fetch related information. With 100 students, this example can produce 1 query for students + 100 enrollment-count queries = 101 queries.

NPlusOne_Bad.cs C# • EF Core

```
1  var students = context.Students.ToList();
2
3  foreach (var student in students)
4  {
5  int count = context.Enrollments
6  .Count(e => e.StudentId == student.StudentId);
7  }
```

Each Count() executes a database query for that student.

Preferred approach: Include()

Include_Good.cs C# • EF Core

```
1  var students = context.Students
2  .Include(s => s.Enrollments)
3  .ToList();
4
5  foreach (var student in students)
6  {
7  Console.WriteLine(
8  $"{student.FirstName} {student.LastName} - " +
9  $"Enrollments: {student.Enrollments.Count}");
10 }
```

Include() eager-loads the related collection. EF Core commonly uses a LEFT JOIN for a simple one-to-many relationship, but split-query settings and query shape can change the SQL/query count.

SQL shape comparison

NPlusOne.sql C# • EF Core

```sql
-- N+1: first load students
SELECT * FROM Students;

-- Then one query per student
SELECT COUNT(*) FROM Enrollments
WHERE StudentId = @StudentId;

-- Typical single-query Include shape
SELECT ...
FROM Students AS s
LEFT JOIN Enrollments AS e
ON s.StudentId = e.StudentId;
```

Why N+1 matters

- Many round trips between the application and database.
- Slower screens and reports as the number of parent rows grows.
- Extra database load and repeated work.

Include() is not always the best answer

If the UI only needs a count or a few fields, loading every related entity may transfer more data than necessary. Projection can return exactly the values needed. Include() also does not guarantee exactly one SQL statement in every query shape.

Projection_Alternative.cs C# • EF Core

```csharp
var report = context.Students
.Select(s => new
{
s.FirstName,
s.LastName,
EnrollmentCount = s.Enrollments.Count()
})
.ToList();
```

For a count-only dashboard, projection is often more efficient than loading the full Enrollments collection.

Inspecting generated SQL

ToQueryString() previews the SQL shape for an IQueryable query; it does not execute it. Runtime logging through LogTo() shows SQL actually executed. Use runtime logs to verify whether the scenario produces one query or repeated queries.

Practical checklist

- Load related data intentionally when it is needed.
- Watch for database calls inside loops.
- Use Include() when you need related entity objects or collections.
- Prefer projection when you only need selected fields or aggregate values.
- Verify SQL and query count using EF Core logging.

Remember: Include() can help avoid N+1 queries, but choose between Include() and projection based on what the application needs.